ICGA Journal -1 (2020) 1–12
DOI 10.3233/ICG-190141
IOS Press

A description language for chess

Manuel Cristóbal López-Michelone [a,*] and Jorge L. Ortega-Arjona [b]
[a] *Posgrado en Ciencia e Ingeniería de Computación, UNAM, México*
[b] *Departamento de Matemáticas, Facultad de Ciencias, UNAM, México*

Abstract. The game of chess involves patterns. In this article we develop a simple description language for chess positions as patterns. It seems useful to catalogue similar positions in a generic form, based on common elements, in order to simplify the search for chess positions. A simple chess pattern language is used here to develop a computer program to find similar chess configurations. The aim is to avoid a commonly-used brute force approach. Further, this language could explain why some typical maneuvers actually work in some patterns. This could also be a step forward to model and build chess programs, which perform in a way closer to what a human being does.

CORRECTED PROOF

1. INTRODUCTION

Chess is a game in which the configurations of the pieces between the two players frequently repeat. These positions share common elements that we generically call *patterns*. Curiously, these elements often occur in positions that appear to be different. Nevertheless, a closer analysis shows that there are common elements that repeat. And just as these configurations are repeated, the way of dealing with them is also repeated, which is interpreted as "making the right moves".

Chess literature is crowded with books in which the authors provide tactical exercises on some already-found chess patterns. These exercises hand over an advantage to the student-player sincee, since without the need to make a complex analysis, the novice player can profit from the experienced knowledge about the next movements that lead to a win or, at least, the moves that guarantee an advantage.

Some studies have shown that the human brain – with constant study and practice – begins to generate a mental catalogue of winning chess patterns. Normally these can sum up to around 50,000 positions (Chase and Simon, 1973). However, no one knows how the brain recognises the chess patterns that look different.

On the other hand, computers are not programmed to play chess with a pattern-oriented system; rather they take advantage of their high-speed computations. Hence, this allows them to do position analysis based on an evaluation function that reviews all possible variations that can follow, using a decision tree approach. However, this does not take advantage of neither the nature of the game of chess, nor of the patterns existing in the game, which have been known for many years.

Here, we propose a language for chess positions, which allows us to define positions as patterns with common elements. This language attempts to show which are the movements of the pieces, or the maneuvers, aiming to take advantage during the game. By making use of such a description language, computer programs can be oriented so they do not require to analyse, withan enormous

*Corresponding author. E-mail: morsa@la-morsa.com.

1389-6911/20/$35.00 © 2020 – IOS Press and the authors. All rights reserved

depth, a decision tree when the pieces are in a known pattern. A pattern language should be flexible enough to find common elements, even if positions seem to be very different.

2. THE DESCRIPTION OF CHESS PATTERNS

There are several ways of describing positions in chess. One of the most popular is called FEN (*Forsyth-Edwards Notation*). Proposed in 1883, unlike notation systems for plays and games, FEN is used to score a position on the chessboard. However, it defines where each piece is unambiguously found on the chessboard.

A pattern description language requires more information about the position. It deals with the most basic elements of chess: attack and defense. A chess player sees when a piece is attacked or when a piece is defended. Nonetheless, it is as well necessary to know whether a piece is able to also attack a set of squares that not necessarily contain any piece. In fact this is how players remember positions. This has been proven, first by Adrian De Groot (de Groot, 1965), and later by H. Simon and B. Chase (Simon and Chase, 1973), who concluded that the player encodes positions as a set of relations between the pieces, instead of seeing positions as if they were photographs. De Groot experimented with the world best players by proposing to them positions which lacked any chess logic, for instance, game states highly unlikely and even illegal chess positions (pawns in the first line, both kings in check, etcetera, in the Simon and Chase study). The great players experienced the same difficulties, just like any chess beginner player to recall these positions. The Simon and Chase analysis and De Groot's study eliminate the myth that the chess player has a photographic memory of the board and, on the other hand, it is confirmed that in some way the players remember the positions as the relationships between the chess pieces.

A chess pattern is thus defined as a generalization of a relationship among pieces (attacks and defenses), containing its essential characteristics. We think this is a novel part in the description of a pattern. Let us consider only two types of relationships: attack and defense. An **attack** occurs when a player's piece assails a piece belonging to the opposite player; a **defense** is given when a piece of one of the players covers another of the same side. Notice that these simple definitions are based on relationships among rival pieces in the attack, and relationships among pieces of the same colour in the defense. Further, the description of the chess patterns goes beyond simply seeing the positions of the pieces in specific squares, by describing them also in terms of what pieces attack and/or defend pieces and the various squares of interest. But this definition is static. A pattern also has to describe the dynamic change in position, which occurs when describing the movement, or sequence of movements that provide an advantage to a player.

In fact, we are interested in patterns that are known to give a certain advantage to one of the players. Chess literature is full of tactical exercises where chess combinations are found based in known themes, like the *greek gift* or the *Lasker–Bauer pattern*, for example. Of course, a representation language could search for other patterns and look for viability of some common characteristics and features of chess positions (see Section 6 below). Right now we are experimenting with patterns specified manually by human chess experts.

Consequently, this description language for chess patterns requires both factors:

1. the definition of features, which allows to find the elements common to the chess positions, so that it is possible to compare apparently different positions sharing the common elements (and this is when we talk about a *defined pattern*), and

CORRECTED PROOF

2. the sequence of movements (variations) that must be made to obtain an advantage in a given position.

The position description language here generalises the patterns found in the different chess configurations, thus providing a simple way of defining the common elements in the analysed positions. Once these patterns are defined, it is possible to generate a list of them, which would issue a new approach to chess programs and their way for assessing positions. This means that the programs could guide the analysis of the movements in a way that may save time, either eliminating the moves that are not promising, or reinforcing the analysis on movements that could be winning positions, or even acquire a tangible advantage from the defined patterns.

3. RELATED WORK

The problem of describing the positions in terms of their characteristics has been analysed by many other authors. For example, Zobrist and Carlson (Zobrist and Carlson, 1973) implemented a chess program that played chess using a series of primitive patterns. The results were promising but for some reason this research line was not continued. In 1990 Michael George wrote his thesis concerning a program called MACH (George and Schaeffer, 1990), which used the idea of *chunks*, that is, blocks defined on the board, which would allow to describe patterns in some way. Chrilly Donninger in 1996 (Donninger, 1996) discovered a way for implementing chess knowledge through a programming language called CHE, which was used in his chess program, *Nimzo*.

But probably the key to this problem lies in the work of Adriaan de Groot, H. Simon and B. Chase, who demonstrated that the chess players do not keep in their memory the most exact positions. Instead, they use the relationships between the pieces on the board.

Paradise (*Pattern recognition applied to directing search*), a knowledge-based chess program written by David Wilkins in 1979–80 (Wilkins, 1979), tried to use chess patterns to find the best moves in tactically sharp middlegames. The software is written in MacLisp and the patterns are written as production rules in an expert system approach, using a Production-Language (Wilkins, 1988). Even the relative success of the idea lacks of a human and simple way to encode the chess patterns.

Finkelstein and Markovitch (Finkelstein and Markovitch, 1998) developed a description language idea, which was used to learn to play selectively using only the patterns of movements in given positions. This work is important because it tries to reduce the problem of dealing with chess positions that are similar on the board.

Of course many commercial and open source chess database programs, like *Chessbase* or *Jose*, can find similar positions using a very limited approach, constructing an exact chessboard position with the pieces involved in the pattern to search. Other interesting approach is *CQL – Chess Query Language* (Costeff, 2004), which tries to find similar chess positions using wildcards queries, but CQL has serious limitations: a CQL user needs to formulate very precisely a query in a very complex query language, and there is not a simple way to measure the similarities of different positions (Ganguly et al., 2014).

Peter W. Frey (Frey, 1986), sought to categorise patterns into three types. This, however, does not seem to have resonated with the programmer community as would have been expected, even from Hans Berliner's *Hi-Tech* "Oracle" approach (Berliner, 1999).

A novel viewpoint can be found in the work by Ganguly and Jones (Ganguly et al., 2014), using an Information Retrieval (IR) approach used commonly to find similarities in image retrieval. In this

CORRECTED PROOF

approach, each position in a chess game is encoded as a set of terms, where each term is used to represent different features of a chess position. Nevertheless, the chess pattern representation presented is not very simple to build by humans.

Other works worth of mention are: "Templates in chess memory: A mechanism for recalling several boards" (Gobet and Simon, 1996). This paper addresses empirically and theoretically a question derived from the chunking theory of memory and it is suggested that chess players use long-term memory retrieval structures or templates in addition to chunks, in short-term memory, to store information quickly.

Gadwal et al. (Dinesh Gadwal and McCall, 1996) investigated a knowledge-based approach to build a prototype chess tutor, which helps students to learn how to play specific bishop-pawn endgames. This idea could be used in our chess representation language to even build some sort of high-level patterns for tutoring tactical chess themes.

On the other hand, P. Ciancarini and M. Gaspari (Ciancarini and Gaspari, 1989) used a very interesting approach to define some sort of interface for middlegame positions in chess through a knowledge base. It looks as a complementary idea to our chess description language which builds a high-level representation of chess positions as human players do. Nevertheless, their motivation is different to our approach but it can be used to build a knowledge-base oriented specifically to chess patterns.

Finally, in Kaneko et al. (Tomoyuki Kaneko and Kawai, 2003), the authors address the problem to find a way to automatic identification of patterns in evaluation functions. The approach in this case comprises three steps: 1. Generation of logic formulae by using the specifications of a target game, 2. Translation of the formulae into patterns, and 3. Selection of a set of suitable patterns from those generated. The authors work with the Othello game and their results are promising. Looks a very interesting way to solve the problem of finding suitable patterns without the need of a human expert in the game.

CORRECTED PROOF

4. DEFINITION OF THE PATTERN LANGUAGE

The description language for chess positions is defined as a number of relationships between pieces, separated by logical connectors, as follows:

relationship [connector] relationship [connector] ... relationship [connector] advice

A relationship (attack or defense) between a piece A and a piece B is written as:

A(B)

where A and B are two chess pieces. In the case of an attack, it is clear that piece A is one colour and piece B is the opposite one. If both are the same colour, then the pattern describes the defense between both pieces. For example, the notation defined as **B(ph7)**, indicates a white bishop[1] which attacks a black pawn located in square h7.

Sometimes it is enough to define a piece attacking an empty square. For example, **B(h7)**, tells us that there is a white bishop attacking square h7.

[1]Uppercase letters represent the white pieces and lowercase ones represent the black pieces.

Also, it is possible to describe where a piece is exactly ont the chessboard. For instance **Bh7** simply indicates that there is a white bishop in the square h7. Note that in the pattern language **B(ph7)**, **B(h7)** and **Bh7** represent three different situations.

It is important to note that the description of chess patterns cannot be seen as positions of pieces in specific squares necessarily, but what must be described is what pieces attack/defend the squares that interest us.

In addition to describing the pattern that contains the fundamental elements of a position that can give the advantage, one must describe at least the first move that must be made to achieve this.

4.1. Summary of the chess description language

The instructions of the language are:

- **A(B)** – Piece A attacks/defends Piece B.
- **A(square)** – Piece A attacks square.
- **A square** – Piece A at square.
- **taboo(square)** – Make a square unavailable to defender.
- **action(chess movement)** – The recommended move for the pattern.
- **structw([list of pawns])** – Defines the structure of the white pawns ont the board.
- **structb([list of pawns])** – Defines the structure of black pawns on the board.
- **Logical connectors(“,”, “;”)** – “,” is AND. “;” is OR. The word “AND” or “OR” may be used instead.
- **Single line comments** can be made with the “//” delimiter, for example, *//This is a comment*.

Uppercase letters denote white pieces (**K**, **Q**, **B**, **N**, **R**, **P**) and lowercase letters denote black pieces (**k**, **q**, **b**, **n**, **r**, **p**).

4.2. Examples of the chess description language

Generally speaking, a chess pattern described in this language is a set of pieces attacking or defending each other, where one can find the position’s common elements. In some cases, all the elements can be present, in other cases only a subset of the elements are present in the position. But in any case, the pattern may show many similarities.

The following example seems to be a way to describe similar positions:

Pattern: Q(h5), B(ph7), B(pg7), R(f3), action(1. Bxh7+) (Figure 1)
Pattern: Q(h5), B(ph7), B(pg7), R(c3), action(1. Bxh7+) (Figure 2)

The patterns defined in the two previous positions are essentially the same. It is clear that the description of each chess position is not strictly the same for the two of them, but the similarities are evident for any chess player. Therefore, it must be pointed out that our description language that can (or at least may attempt to) deal with positions that are not necessarily the same, and still very similar.

Our approach is to find a way to characterise chess positions as patterns. According to Peter W. Frey (Frey, 1986), three types of sets are proposed: *Type-A*, where the characteristics of the pattern are completely fulfilled; *Type-B*, where the characteristics sought are usually present, but not always present; and finally *Type-C*, whose characteristics are occasionally present. This, in turn, allows chess

CORRECTED PROOF

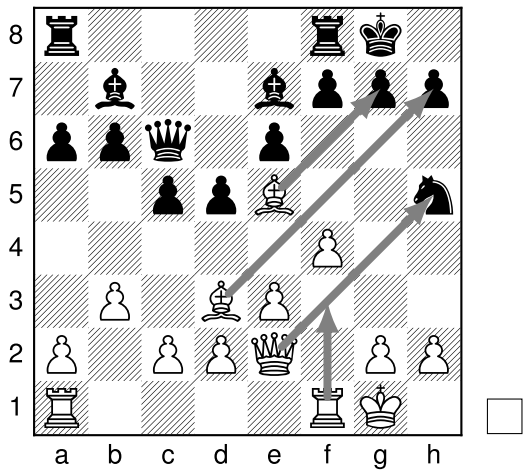

Fig. 1. **Lasker–Bauer**, Amsterdam 1889.

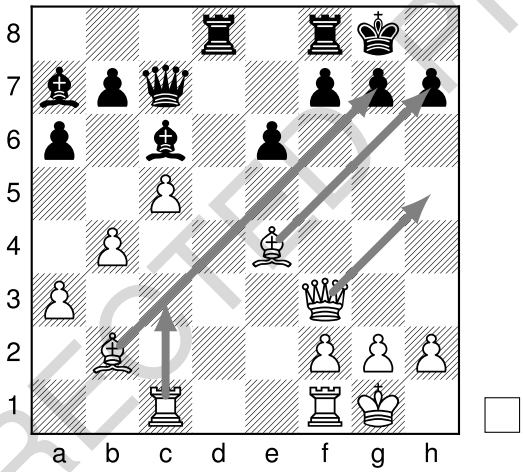

Fig. 2. **Miles, A. – Browne, W.**, Luzern (ol) 1982.

CORRECTED PROOF

programs to be designed as in the case of *Nimzo* (Donninger, 1996) or *Hi-Tech* program (Berliner, 1999), in such a manner that the definition of the chess knowledge can change the behaviour of the chess program.

The language also must deal with prohibited squares. For instance, in the last match of the World Championship (New York 2016), Carlsen found a winning combination in the following position:

Pattern: Q(h6), Rc8, R(f7), Ph5, kh7, pg7, taboo(g6), action(1. Qh6+) (Figure 3)

In this pattern, **taboo(g6)** means that square g6 is not available to the defender (meaning in this particular case, the king is not allowed to move to g6).

But this same pattern already appeared in **Popov–Novopashin**, USSR Championship 1979:

Pattern: Q(h6), Rf8, R(b7), Ph4, kh7, pg7, taboo(g6), action(1. Qh6+) (Figure 4)

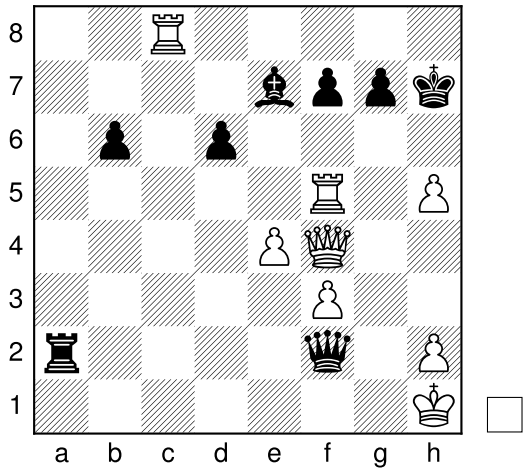

Fig. 3. **Carlsen, M. – Karjakin, S.**, World Chess Championship 2016.

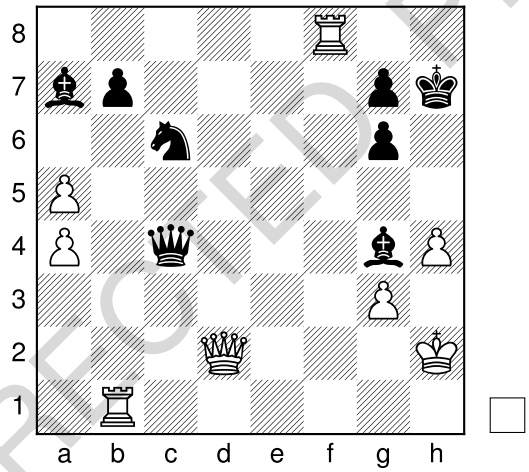

Fig. 4. **Popov, N. – Novopashin, A.**, URS (ch) 1979.

CORRECTED PROOF

Notice here the black king cannot go to g6 because of his own pawn. This is another way to make square g6 unavailable to Black at the definition stage of the pattern.

4.3. Logical connectors

A pattern consists of a sequence of symbols defining the chess pieces in certain squares (or attacking other pieces) connected logically. The pattern can be considered as a set of statements connected by the AND logic connector (this is the default condition). We can also use the OR connector, which is used when a chess piece in the pattern, could be in that square or in another one. In the best Prolog language tradition, we use the "," symbol for the AND and ";" for OR, but it can be written simply "AND" or "OR".

4.4. Positional patterns

A position description language should deal not only with tactical elements. It should have the capability to define positional patterns. Most of these patterns depend on the pawn structure for both players. By defining **structw([list of pawns])** for white and **structb([list of pawns])** for black, and the advice of what to do, the pattern can be constructed. Russian chess literature called them *priyomes*.[2]

Positional patterns, *aka* priyomes, are more difficult to define because there is generally no straightforward method to get the advantage. But, in some situations, there is a way to define a pattern.

A very popular priyome is the explotation of a weak square, for example c6. In **Botvinnik–Donner**, Amsterdam 1963 (see next diagram), the c6 square is weak. The steps to get the advantage are: (a) trade light-square bishops, (b) secure c6 with b4–b5, and (c) occupy the hole with a knight.

The chess pattern language describes this with pawns structures for both players:

structw(a3, b4, e2, f2, g3, h2), structb(a7, b6, e6, f7, g7, h7), (Figure 5)

with the advice: **action(get c6 for white)**, using the steps mentioned above.[3]

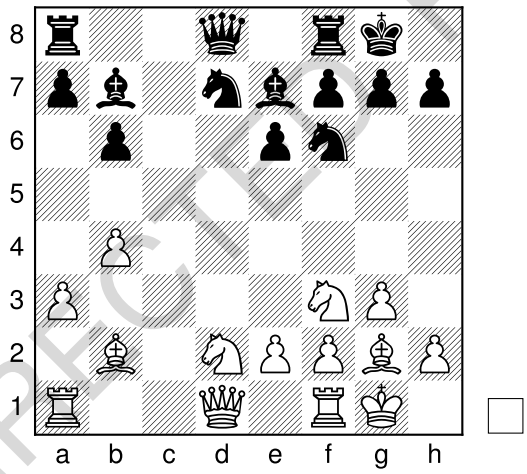

Fig. 5. **Botvinnik, M. – Donner, J.**, Amsterdam 1963.

A second example of the same pattern: In **Aronian–Carlsen**, Elista 2007:

structw(a3, b4, e3, f2, g2, h2), structb(a7, b6, e6, f7, g7, h6), (Figure 6)

with the advice: **action(get c6 for white)**.[4]

[2]In chess literature players use "priyomes", a Russian noun that is used directly and generically in English, to represent some sort of typical maneuver or technique in chess (i.e. a pattern). For example, a typical defensive technique ("priem") in rook endings is to use the king to attack the opponent's pawns. The Russian word "priem" is used more generally to refer to typical maneuvers used in positions with certain pawn structure or other defining characteristics. (http://chess-training.blogspot.mx/2016/05/priyomes.html).

[3]Botvinnik accomplish the plan with: 14. Nd4 Bxg2 15. Kxg2 Qc7 16. Qb3 Rfc8 17. Rfc1 Qb7+ 18. Qf3 Nd5 19. e4 N5f6 20. b5 a6 21. Nc6 and white is much better.

[4]Aronian followed the typical steps: 16. Ba6 Bxa6 17. Qxa6 Rxc1+ 18. Rxc1 Nb8 19. Qc4 Rd8 20. h3 Ne8 21. b5! and eventually white won the game.

CORRECTED PROOF

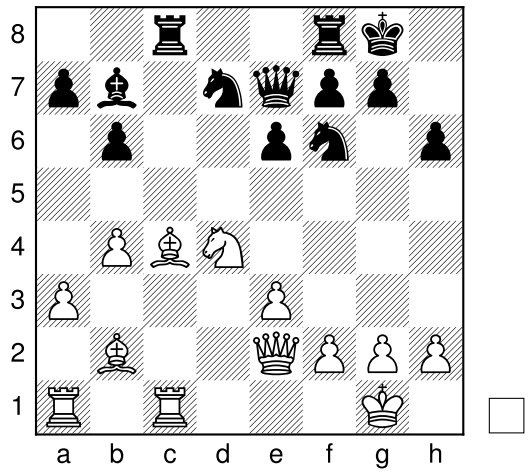

Fig. 6. **Aronian, L. – Carlsen, M.**, Elista (Candidates) 2007.

The pawn structures shown for each player are not exactly the same, but the weakness of the "c6" square is obvious for a chess expert, and the priyome is part of what chess masters know about weak squares in chess pawn structures. We are not dealing in this case with an automatic recognition of chess patterns. Again, pawn structures do not have to be exact, but just similar in some key aspects.[5] So positional patterns are chess positions without a tactical conclusion.

5. EXPERIMENTAL DATA AND ANALYSIS

We built a computer program to find chess patterns using our chess description language. We use the *Megabase 2018* from *Chessbase* in Portable Game Notation (PGN) format,[6] a commercial collection of chess games (7,187,893 games, years 1475 to 2017), and, as an example, we search for the greek gift pattern (for White), defined as kg8, pf7, pg7, B(ph7), Nf3, Qd1, Pe5.

We find 1194 games within the *greek gift*:

1. 99 games finish in a draw (8 percent). We are going to dismiss these games because are equal for both, Black and White.
2. White win in 907 games (76 percent)
3. Black win in 188 games (16 percent)
4. Not all games are from rated chess players. In 700 rated games, White has 1971 (average) Elo points[7] with a performance of 2097 Elo points. This means, in an average of 1971 Elo points, White have 126 Elo points above the expectancy of the results, showing the effectiveness of the greek gift.
5. On the other hand, in 840 rated games, Black has 1823 (average) Elo points with a performance of 1756 Elo points, meaning a bad performance for Black, below of the expectancy of the results.

[5] In the example, the key aspect is the weak square c6.

[6] http://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm

[7] Elo is a numerical value of the chess level of a player. A beginner has 1000 Elo points and the World Champion usually is around 2850 Elo points. See http://gobase.org/studying/articles/elo/.

CORRECTED PROOF

Fig. 7. The pattern's chess program.

A first conclusion one gets at is the effectiveness of the greek-gift pattern, shown in 76 percent of the games in the sample. This means we can guide the search of the best moves using the chess pattern found instead of analysing all the possible moves in a specific position.

Our experimental application can find any pattern defined in our chess description language. The patterns can be written using any ASCII text editor. The application works in the Microsoft Windows UI.[8] The program is CPU intensive because it has to check every position in every game and compare it against the pattern defined by the user.

We tested our program on two computers. The first one is an AMD FX(tm)-8350 Eight-Core processor, running at 4.0 Ghz, with 4 GBytes of RAM using Windows 7 Pro (64 bits); and a second computer with an AMD Ryzen 5 PRO 1600 Six-Core Processor (64 bits), at 3.20 GHz with 16 GBytes of RAM, using 64 bit Windows 10 Pro version. In the fastest version (Windows 10), the software can find a pattern (with some optimisations done), analysing all the positions generated in a game and comparing with the target pattern, in around one second per game. Of course, the software needs more optimisations. Right now we are not using any sort of database, for the PGN games, that could make the search faster. Our prototype software is just a proof of concept.

[8]Tested on Windows 7 and Windows 10 without trouble.

In a scenario of a chess program searching for patterns, one second to find if a pattern exists in a game is very slow, but the imaginary chess program would be working using tested patterns (defined, in principle, by chess masters) and it could be embedded in the evaluation function in some way, in an attempt to speed the calculations.

6. CONCLUSION AND FUTURE WORK

A language for describing patterns of chess positions may be an interesting approach to try to solve the problem of the growth of the tree of variations to be analysed. A language of this nature can optimize the search of the best moves without having to check many variations that are not useful. In fact this is the way chess masters play.

By means of this position description language, an important number of known chess patterns can be build to make a catalogue of them, which may lead to the classification of many positions in chess which will not require an analysis of brute force. One effort to clasify middlegame patterns can be found in the Encyclopedia of Chess Combinations (by Chess Informant), but the classification of middlegame tactical themes is very ambiguous.[9]

This is a first step, although it is clear that we still have to analyse the problems of similarity and how far a pattern can appear in a given position and how similar it is to the configuration we are looking at. This approach has not been solved at this time, and it could be part of future work on the subject.

The proposed language allows to start with description patterns that are very well-known in chess literature in which we know how to solve them. Nonetheless, a new approach could be to build a neural network to try to find other patterns or even different sequences of moves to solve the found patterns, that is, to get the advantage in the game. This could also be part of future work in the subject.

Automatic recognition of chess patterns could be a novel approach, and probably the best tool available could be some sort of neural network. In our approach, we have to define the chess patterns drawing from the expertise of chess masters.

Also, in the present form, the user has to define patterns for White or Black pieces separately, but we have plans to add some syntactic constructions to search indistinctly the same pattern for White and Black. This will be part also of the representation language.

CORRECTED PROOF

REFERENCES

Berliner, H. (1999). *The System*. Gambit Books.

Chase, W. & Simon, H. (1973). The mind's eye in chess. In *Visual Information Processing* (pp. 215–281). doi:10.1016/B978-0-12-170150-5.50011-1.

Ciancarini, P. & Gaspari, M. (1989). A knowledge based system and a development interface for the middle game in chess. In *Advances in Computer Chess* (Vol. 5).

Costeff, G. (2004). The chess query language: CQL. *ICGA Journal*, *27*(4), 217–225. doi:10.3233/ICG-2004-27404.

de Groot, A. (1965). Thought and choice in chess. *Mounton.*

[9] https://sahovski.com/Encyclopedia-of-Chess-Combinations-5th-edition-p92256834

Dinesh Gadwal, J.E.G. & McCall, G.I. (1996). Tutoring bishop-pawn endgames: An experiment in using knowledge-based chess as a domain for intelligent tutoring. *Applied Intelligence*, *3*, 207–224.

Donninger, C. (1996). CHE: A graphical language for expressing chess knowledge. *ICCA Journal*, *19*(4), 234–241.

Finkelstein, L. & Markovitch, S. (1998). Learning to play chess selectively by acquiring move patterns. *ICCA Journal*, *21*(2), 100–119.

Frey, P.W. (1986). Fuzzy production rules in chess. *ICCA Journal*, *9*(4), 175–179.

Ganguly, D., Leveling, J. & Jones, G.J. (2014). Retrieval of similar chess positions. In *Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 687–696).

George, M. & Schaeffer, J. (1990). Chunking for experience. *ICCA Journal*, *13*, 123–132.

Gobet, F. & Simon, H.A.R. (1996). Templates in chess memory: A mechanism for recalling several boards. *Cognitive Psychology*, *31*, 1–40. doi:10.1006/cogp.1996.0011.

Simon, H. & Chase, B. (1973). Perception in chess. *Cognitive Psychology*, *4*, 55–81. doi:10.1016/0010-0285(73)90004-2.

Tomoyuki Kaneko, K.Y. & Kawai, S. (2003). Automated identification of patterns in evaluation functions. *IFIP Advances in Information and Communication Technology*, *135*, 279–298.

Wilkins, D. (1979). Using patterns and plans to solve problems and control search. PhD thesis, Computer Science Dept, Stanford University, AI Lab Memo AIM-329.

Wilkins, D. (1988). Using patterns and plans in chess. In *Computer Chess Compendium* (pp. 233–257). doi:10.1007/978-1-4757-1968-0_23.

Zobrist, A. & Carlson, F. Jr. (1973). An advice-taking chess computer. *Scientific American*, *228*(6), 92–105. doi:10.1038/scientificamerican0673-92.

CORRECTED PROOF